

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
UTEQ

Carrera de Ingeniería en Sistemas

TALLER: MODELADO DE COMUNICACIÓN ENTRE PROCESOS DISTRIBUIDOS

FORM-TL-01

*Sistema Bancario y de Notificaciones de Bolsa:
“BankPulse & Stock-Ticker”*

Integrantes:

Cristhian Daniel Pacheco Cárdenas

Ernesto Gregory Luna Mora

Robinson Rodrigo Cando Moreno

Materia: Aplicaciones Distribuidas

Año: 2026

Índice

1. Resumen Ejecutivo	2
2. Introducción	2
3. Marco Teórico	3
3.1. Sistemas distribuidos y comunicación entre procesos	3
3.2. Paradigmas de comunicación entre procesos	3
3.3. Middleware: concepto y capas de abstracción	4
4. Escenario de Negocio: Sistema Bancario “BankPulse & Stock-Ticker”	4
4.1. Contexto y justificación	4
4.2. Descripción de los módulos	5
5. Diseño y Modelado de la Arquitectura	5
5.1. Arquitectura implementada (capa de sockets)	5
5.2. Arquitectura objetivo extendida (incorporando RPC y mensajería)	6
6. Selección de Protocolo y Middleware: Justificación de Diseño	6
7. Implementación Técnica	7
7.1. Módulo transaccional TCP — Servidor	7
7.2. Módulo transaccional TCP — Cliente	9
7.3. Módulo de notificaciones UDP — Servidor	10
7.4. Módulo de notificaciones UDP — Cliente	11
8. Resultados y Verificación Experimental	11
9. Notas de Presentación Oral (Guion de Defensa)	13
9.1. Puntos clave a comunicar	13
9.2. Posibles preguntas críticas y respuestas anticipadas	13
10.Repositorio del Proyecto	14
11.Declaración de Uso de Inteligencia Artificial	14
12.Conclusiones	14
13.Referencias Bibliográficas	14

1. Resumen Ejecutivo

El presente informe documenta el diseño y la implementación de un sistema distribuido que modela la comunicación entre procesos dentro de un escenario de negocio bancario. El sistema, denominado “BankPulse & Stock-Ticker”, está compuesto por dos módulos funcionalmente diferenciados: un módulo transaccional que utiliza el protocolo TCP para garantizar fiabilidad en operaciones críticas (consultas de saldo y confirmaciones de depósito), y un módulo de notificaciones bursátiles que utiliza el protocolo UDP para priorizar la velocidad de entrega sobre la garantía de recepción.

Esta versión del documento corrige y profundiza una entrega previa, incorporando el diagrama de arquitectura solicitado por el taller, un marco teórico sobre los paradigmas de comunicación entre procesos distribuidos (sockets, llamada a procedimiento remoto, invocación de métodos remotos y mensajería orientada a mensajes), una tabla comparativa que justifica la selección de protocolo y middleware en función del escenario de negocio, y el guion de defensa oral exigido por la actividad. Se mantiene como evidencia de base la implementación funcional en Java de los servidores y clientes TCP/UDP, la cual sirve como prueba de concepto del nivel de comunicación más bajo de la arquitectura propuesta.

2. Introducción

La comunicación entre procesos es uno de los pilares conceptuales de los sistemas distribuidos: sin un mecanismo confiable para que componentes ubicados en distintos nodos de red intercambien información, no existiría la posibilidad de construir aplicaciones que se ejecuten de forma coordinada sobre múltiples máquinas. Antes de escribir una sola línea de código, la ingeniería de software exige modelar dicha comunicación: decidir qué paradigma se usará (sockets, llamadas remotas, mensajería), qué protocolo de transporte es adecuado para cada flujo de datos, y qué capas de middleware abstraerán la complejidad subyacente del sistema operativo y de la red.

El presente trabajo tiene como objetivo modelar una comunicación para un escenario bancario asignado por el docente, partiendo de una base de implementación ya construida en una entrega anterior. El documento sigue una estructura comparable a la de un trabajo de investigación preliminar: parte de un marco teórico, plantea el problema dentro de un escenario de negocio concreto, propone y justifica un diseño arquitectónico, y cierra con la evidencia de implementación y las conclusiones correspondientes.

3. Marco Teórico

3.1. Sistemas distribuidos y comunicación entre procesos

Un sistema distribuido puede entenderse, de forma sencilla, como un conjunto de computadoras independientes que se presentan ante el usuario final como un único sistema coherente. Para que esa coherencia exista, los procesos que corren en nodos distintos necesitan un mecanismo de comunicación entre procesos (*Inter-Process Communication*, IPC) que les permita intercambiar datos a través de la red como si estuvieran intercambiando datos en memoria local, aunque con las limitaciones propias de la latencia, la pérdida de paquetes y la posibilidad de fallos parciales [1, 2].

3.2. Paradigmas de comunicación entre procesos

Existen, en términos generales, cuatro paradigmas ampliamente utilizados para resolver la comunicación entre procesos distribuidos. A continuación se describe cada uno de forma conceptual

Sockets Es el paradigma de más bajo nivel de los cuatro. Un *socket* es un punto final de comunicación identificado por una dirección IP y un número de puerto, sobre el cual los procesos intercambian flujos de bytes (TCP) o datagramas independientes (UDP). Es el paradigma sobre el cual, en la práctica, suelen construirse los otros tres.

Llamada a procedimiento remoto (RPC) Permite que un proceso invoque una función que se ejecuta físicamente en otra máquina, ocultando al programador los detalles de serialización y transporte de red mediante un *stub* en el cliente y un *skeleton* en el servidor. Implementaciones modernas como gRPC utilizan esquemas de definición de interfaz fuertemente tipados (Protocol Buffers) para garantizar compatibilidad entre lenguajes.

Invocación de métodos remotos (RMI) Es una especialización de RPC propia del ecosistema Java, en la cual lo que se invoca de forma remota no es una función aislada sino un método de un objeto remoto registrado en un *RMI Registry*. Mantiene la orientación a objetos del lenguaje, a costa de quedar limitado a aplicaciones que estén escritas en Java en ambos extremos.

Mensajería orientada a mensajes (MOM) A diferencia de los tres paradigmas anteriores, que son predominantemente síncronos, la mensajería desacopla al emisor del receptor mediante una cola o un tópico intermedio (por ejemplo, RabbitMQ o Apache Kafka). El emisor publica un mensaje sin necesidad de que el receptor esté disponible en ese instante, lo cual resulta valioso cuando no se requiere una respuesta inmediata.

3.3. Middleware: concepto y capas de abstracción

Se entiende por *middleware* a cualquier capa de software que se ubica entre el sistema operativo y la aplicación, con el propósito de ocultar la complejidad de la comunicación en red. En este trabajo se identifican dos niveles de middleware aplicados a la implementación actual:

1. **Nivel de transporte:** la API de *Sockets* de Java (`java.net`), que abstrae el manejo de los sockets del sistema operativo (*bind*, *listen*, *accept*) detrás de clases como `ServerSocket`, `Socket` y `DatagramSocket`.
2. **Nivel de concurrencia:** el `ExecutorService` de Java, que abstrae la creación y destrucción de hilos detrás de un patrón de *Thread Pool*, evitando que el programador gestione manualmente el ciclo de vida de cada hilo.

Como se desarrollará en la Sección 6, una arquitectura bancaria de nivel productivo normalmente incorporaría además middleware de los otros dos niveles mencionados (RPC y mensajería), aunque dicha incorporación queda, en esta entrega, en el plano del diseño propuesto y no de la implementación.

4. Escenario de Negocio: Sistema Bancario “BankPulse & Stock-Ticker”

4.1. Contexto y justificación

Las instituciones financieras modernas requieren infraestructuras de software capaces de atender simultáneamente dos flujos de datos con requisitos contrapuestos: por un lado, operaciones bancarias donde la integridad del dato es innegociable; por otro, notificaciones de mercado donde el valor de la información decae con el paso de los milisegundos.

Cuadro 1: Requisitos contrapuestos del escenario BankPulse & Stock-Ticker

Característica	Transacciones bancarias	Notificaciones de bolsa
Prioridad	Fiabilidad sobre latencia	Latencia sobre fiabilidad
Naturaleza del dato	Crítico (saldo, depósito)	Perecedero (precio de mercado)
Tipo de conexión	Persistente (con <i>handshake</i>)	Sin conexión (datagramas)

4.2. Descripción de los módulos

Módulo de transacciones (TCP). Simula tres operaciones críticas del dominio bancario, expresadas mediante comandos de texto: consulta de información de cuenta (**HORA**), confirmación de una operación de depósito (**ECO**), y cierre ordenado de sesión (**SALIR**). El requisito de diseño es que ningún dato bancario se pierda ni se entregue fuera de orden.

Módulo de notificaciones de bolsa (UDP). Modela el envío de actualizaciones de precio entre el servidor y un monitor de bolsa (*stock ticker*). El requisito de diseño es minimizar la latencia de entrega, asumiendo que la pérdida ocasional de un datagrama es aceptable porque la siguiente actualización lo sustituye casi de inmediato.

Nota metodológica: en la presente entrega, los comandos **HORA** y **ECO** actúan como una simulación simplificada de las operaciones de saldo y depósito —retornan un dato verificable (la hora del servidor) en lugar de un saldo real— con el fin de demostrar el comportamiento del canal de comunicación. La lógica de negocio bancaria completa (cálculo real de saldos, persistencia de transacciones) queda fuera del alcance de este taller formativo y se reserva para la fase de implementación de la PE-U1.

5. Diseño y Modelado de la Arquitectura

5.1. Arquitectura implementada (capa de sockets)

La Figura 1 presenta el diagrama de comunicación correspondiente a la implementación actual: un patrón cliente-servidor clásico con dos canales independientes (puerto 9000 para TCP y puerto 9001 para UDP), un *pool* fijo de diez hilos para gestionar la concurrencia, y una capa lógica que procesa las solicitudes de ambos protocolos.

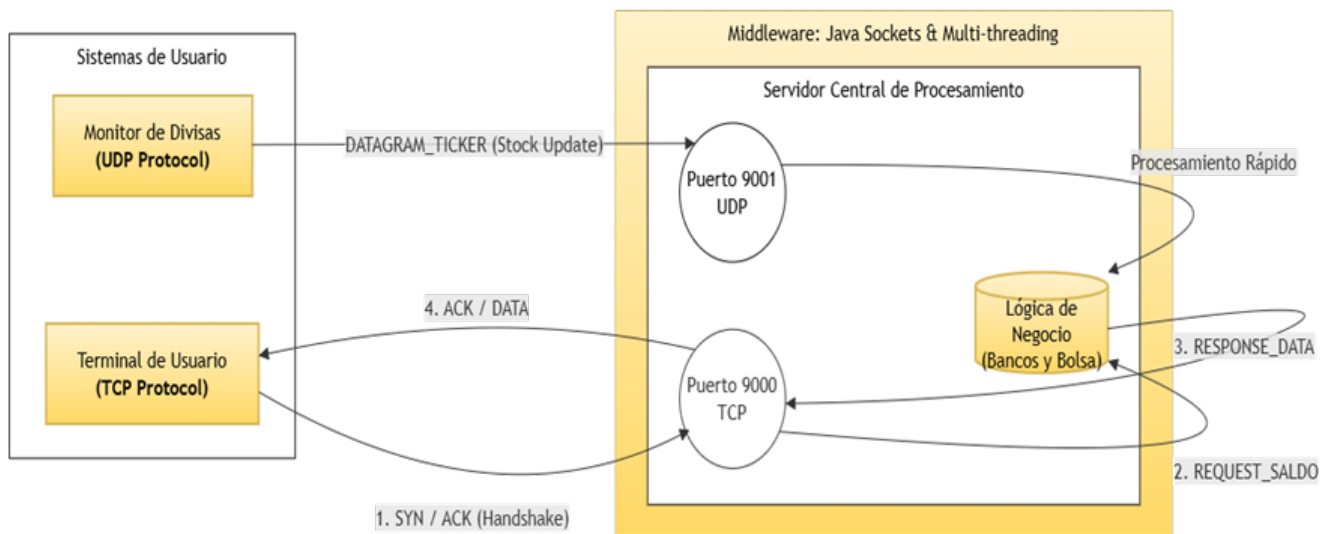


Figura 1: Diagrama de arquitectura de la capa de sockets implementada (TCP/UDP).

5.2. Arquitectura objetivo extendida (incorporando RPC y mensajería)

Atendiendo a la observación recibida en la retroalimentación formativa de la entrega anterior, se propone a continuación una arquitectura objetivo que extiende la capa de sockets ya implementada hacia un esquema de microservicios más representativo de un sistema bancario real. En este diseño propuesto:

- Un **API Gateway** (HTTP/REST) recibe las solicitudes del cliente.
- El **Servicio de Autenticación** se comunica mediante sockets TCP, porque requiere una respuesta síncrona y confiable antes de autorizar cualquier operación posterior.
- El **Servicio de Transacciones** se comunica mediante RPC (gRPC), porque se beneficia de un contrato de interfaz fuertemente tipado entre microservicios.
- El **Servicio de Notificaciones** se comunica mediante mensajería asíncrona mediante un *broker* tipo RabbitMQ, porque el usuario no necesita esperar a que la notificación se entregue para continuar su sesión.

Se aclara que esta segunda arquitectura corresponde a una propuesta de diseño y no a una implementación construida en el código fuente de esta entrega; se incluye porque el taller exige identificar y justificar los cuatro paradigmas de comunicación, y no únicamente el paradigma de sockets sobre el cual se construyó la prueba de concepto.

6. Selección de Protocolo y Middleware: Justificación de Diseño

Esta Tabla 2 resume, para cada paradigma de comunicación identificado en el marco teórico, el middleware típico asociado, su modelo de interacción y el rol que ocupa (o podría ocupar) dentro del escenario BankPulse.

Cuadro 2: Comparación de paradigmas de comunicación aplicados al escenario BankPulse & Stock-Ticker

Paradigma	Middleware típico	Modelo	Rol y justificación en BankPulse
Sockets TCP	API <code>java.net</code>	Síncrono, con conexión	Implementado. Garantiza orden de bytes y retransmisión automática; obligatorio para datos bancarios donde no se tolera pérdida ni desorden.
Sockets UDP	API <code>java.net</code>	Síncrono, sin conexión	Implementado. Reduce la latencia al eliminar el <i>handshake</i> ; adecuado porque un dato de precio perdido queda obsoleto de inmediato.
RPC / gRPC	<i>Stubs</i> generados, Protocol Buffers	Síncrono, llamada remota tipada	Propuesto. Se reservaría para la comunicación entre el servicio de transacciones y otros microservicios internos, donde conviene un contrato de interfaz estricto.
RMI	<i>RMI Registry</i>	Síncrono, orientado a objetos	Descartado. Aporta poco frente a RPC en un contexto donde no todos los componentes futuros estarían escritos en Java.
Mensajería (MOM)	RabbitMQ / Kafka	Asíncrono, publicación-suscripción	Propuesto. Se reservaría para el servicio de notificaciones, desacoplando el envío de alertas del flujo principal de la transacción.

En síntesis, para el sistema bancario BankPulse se eligió TCP en el módulo transaccional porque la fiabilidad es un requisito no negociable, y UDP en el módulo de notificaciones de bolsa porque la velocidad pesa más que la garantía de entrega. RPC y mensajería se identifican como paradigmas pertinentes para una evolución futura del sistema hacia una arquitectura de microservicios, mientras que RMI se descarta por restringir la interoperabilidad a un único lenguaje de programación.

7. Implementación Técnica

A continuación se presenta el código fuente correspondiente a la capa de sockets, que constituye la prueba de concepto de la arquitectura.

7.1. Módulo transaccional TCP — Servidor

```
public class ServidorTCP {
    private static final int PUERTO = 9000;
    private static final int MAX_HILOS = 10;
    private static final AtomicInteger contadorClientes = new
        AtomicInteger(0);
```



```

public static void main(String[] args) throws IOException {
    ExecutorService pool = Executors.newFixedThreadPool(MAX_HILOS);
    try (ServerSocket servidor = new ServerSocket(PUERTO)) {
        System.out.println "[" + timestamp() + "] Servidor listo en : " + PUERTO);
        while (true) {
            Socket cliente = servidor.accept();
            int id = contadorClientes.incrementAndGet();
            System.out.println "[" + timestamp() + "] Cliente #" + id + " conectado desde " + cliente.getInetAddress());
            ;
            pool.submit(new ManejadorCliente(cliente, id));
        }
    }

    public static String timestamp() {
        return LocalDateTime.now().format(DateTimeFormatter.ofPattern("HH:mm:ss.SSS"));
    }
}

```

7.2. Módulo transaccional TCP — Cliente

```
public class ClienteTCP {
    private static final String HOST = "[DIRECCION-IP-DEL-SERVIDOR]";
    private static final int PUERTO = 9000;

    public static void main(String[] args) throws IOException {
        try (Socket socket = new Socket(HOST, PUERTO);
            BufferedReader entrada = new BufferedReader(new
                InputStreamReader(socket.getInputStream()));
            PrintWriter salida = new PrintWriter(socket.getOutputStream
                (), true);
            Scanner teclado = new Scanner(System.in)) {

            System.out.println("Conectado a " + HOST + ":" + PUERTO);
            System.out.println("Comandos disponibles: HORA | ECO <texto>
                | SALIR");

            String comando;
            while (true) {
                System.out.print("> ");
                comando = teclado.nextLine().trim();
                if (comando.isEmpty()) continue;
                salida.println(comando);
                String respuesta = entrada.readLine();
                System.out.println("Servidor: " + respuesta);
                if (comando.equalsIgnoreCase("SALIR")) break;
            }
        }
        System.out.println("Conexion cerrada.");
    }
}
```

7.3. Módulo de notificaciones UDP — Servidor

```
public class ServidorUDP {
    private static final int PUERTO = 9001;
    private static final int TAM_BUF = 1024;

    public static void main(String[] args) throws Exception {
        try (DatagramSocket socket = new DatagramSocket(PUERTO)) {
            System.out.println("Servidor UDP escuchando en puerto " +
                PUERTO);
            byte[] buffer = new byte[TAM_BUF];
            while (true) {
                DatagramPacket paquete = new DatagramPacket(buffer,
                    buffer.length);
                socket.receive(paquete);
                String mensaje = new String(paquete.getData(), 0, paquete
                    .getLength());
                String hora = LocalDateTime.now().format(
                    DateTimeFormatter.ofPattern("HH:mm:ss.SSS"));
                System.out.println "[" + hora + "] Recibido de " +
                    paquete.getAddress()
                        + ":" + paquete.getPort() + " -> " + mensaje);
                String respuesta = "ACK: " + mensaje + " @ " + hora;
                byte[] resp = respuesta.getBytes();
                DatagramPacket respPkt = new DatagramPacket(resp, resp.
                    length,
                        paquete.getAddress(), paquete.getPort());
                socket.send(respPkt);
            }
        }
    }
}
```

7.4. Módulo de notificaciones UDP — Cliente

```
public static void main(String[] args) throws Exception {
    try (DatagramSocket socket = new DatagramSocket()) {
        socket.setSoTimeout(TIMEOUT);
        InetAddress servidor = InetAddress.getByName(HOST);
        Scanner sc = new Scanner(System.in);
        System.out.println("Cliente UDP. Escribe un mensaje (o 'salir'):"
            );
        while (true) {
            System.out.print("> ");
            String linea = sc.nextLine().trim();
            if (linea.equalsIgnoreCase("salir")) break;
            byte[] datos = linea.getBytes();
            DatagramPacket pkt = new DatagramPacket(datos, datos.length,
                servidor, PUERTO);
            socket.send(pkt);
            byte[] buf = new byte[TAM_BUF];
            DatagramPacket resp = new DatagramPacket(buf, buf.length);
            try {
                socket.receive(resp);
                System.out.println("Respuesta: " + new String(resp.
                    getData(), 0, resp.getLength()));
            } catch (SocketTimeoutException e) {
                System.out.println("Timeout: no se recibio respuesta en "
                    + TIMEOUT + "ms");
            }
        }
    }
    System.out.println("Cliente UDP cerrado.");
}
```

8. Resultados y Verificación Experimental

Las siguientes capturas evidencian la ejecución correcta de ambos módulos.

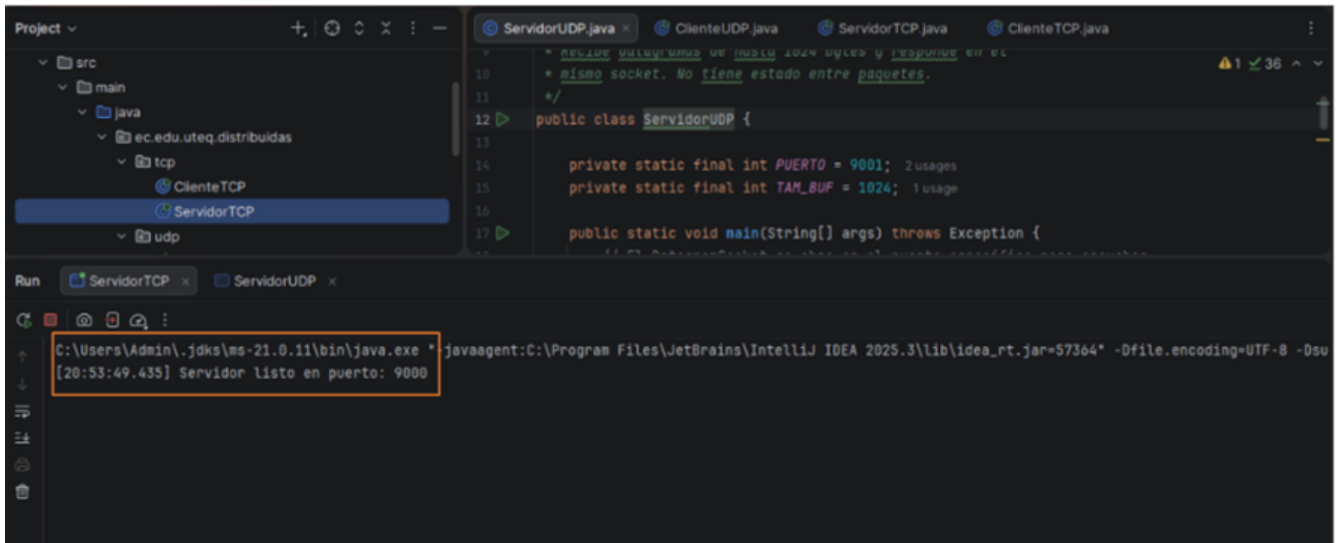


Figura 2: Servidor TCP en ejecución, escuchando en el puerto 9000.

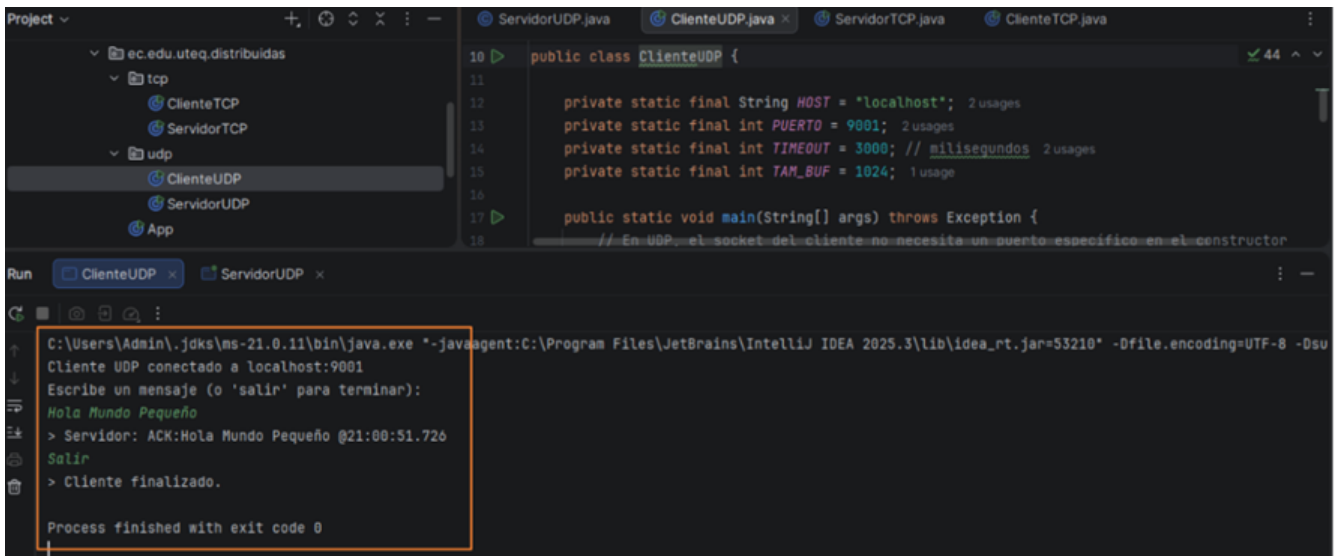


Figura 3: Cliente UDP enviando un mensaje y recibiendo el acuse de recibo (ACK) del servidor.

9. Notas de Presentación Oral (Guion de Defensa)

9.1. Puntos clave a comunicar

Punto 1: Justificación de la arquitectura dual (TCP + UDP). Se debe explicar que se implementaron dos protocolos distintos porque cada uno resuelve un problema diferente: TCP es confiable pero más lento, adecuado para transacciones donde no se puede perder ni un bit; UDP es rápido pero no confiable, adecuado para datos temporales que quedan obsoletos en milisegundos, como los precios de las acciones.

Punto 2: Concurrencia y escalabilidad. El servidor TCP utiliza un `ExecutorService` con un *pool* fijo de diez hilos, de modo que el servidor nunca bloquea a un cliente mientras atiende a otro, y nunca crea hilos de forma indefinida ante una ráfaga de solicitudes, evitando el agotamiento de memoria.

Punto 3: Tolerancia a fallos mediante *timeout*. El cliente UDP configura un tiempo de espera de tres segundos mediante `setSoTimeout()`. Si el servidor no responde en ese intervalo, se lanza una excepción controlada en lugar de bloquear al cliente de forma indefinida, lo cual es un patrón crítico para evitar cascadas de fallos en sistemas distribuidos.

Punto 4: Middleware y capas de abstracción. El middleware actual opera en dos niveles: la API de *Sockets* de Java, que abstrae los detalles del sistema operativo, y el `ExecutorService`, que abstrae la gestión manual de hilos.

9.2. Posibles preguntas críticas y respuestas anticipadas

Dado que cada equipo debe formular al menos una pregunta crítica a los demás, se anticipan las siguientes:

- **¿Por qué no se implementaron RPC o mensajería si los identifican en su análisis?**
Porque el alcance formativo de este taller es modelar el diseño; la capa de sockets implementada sirve como prueba de concepto del nivel más bajo de la arquitectura, mientras que RPC y mensajería se dejan como diseño propuesto para la fase de implementación completa.
- **¿Qué pasa si se pierde un datagrama UDP con el precio de una acción?**
Se acepta la pérdida porque el siguiente datagrama, que llega casi de inmediato, contiene un precio más actualizado; reintentar el datagrama perdido costaría más en latencia que el valor de la información perdida.
- **¿Cómo evitarían que el servidor colapse si llegan miles de clientes simultáneos?**
El *pool* fijo de hilos limita la concurrencia activa; las solicitudes adicionales se encolan en lugar de crear hilos nuevos, aunque se reconoce que un sistema productivo requeriría además balanceo de carga horizontal, fuera del alcance de esta entrega.

10. Repositorio del Proyecto

El código fuente del proyecto está disponible en los siguientes repositorios de GitHub:

- Repositorio: <https://github.com/CristhianP03/Modelado-de-Comunicaci-n-entre-Procesos-Distribuidos>

11. Declaración de Uso de Inteligencia Artificial

En estricto cumplimiento de la rúbrica y las directrices institucionales sobre honestidad académica, se declara formalmente el alcance, las técnicas y la metodología bajo la cual se emplearon herramientas de Inteligencia Artificial (IA) Generativa en este trabajo técnico:

- **Mejora de Redacción y Estructura:** Se emplearon modelos de lenguaje avanzados (LLMs) como soporte para la corrección de estilo y la reorganización del texto. Esto incluyó la mejora del flujo narrativo, la eliminación de frases redundantes, la depuración ortográfica del text

12. Conclusiones

El presente trabajo permitió modelar la comunicación entre procesos distribuidos para un escenario bancario concreto, partiendo de una base de implementación en sockets TCP y UDP, y extendiéndola conceptualmente hacia una arquitectura de microservicios que incorpora RPC y mensajería como paradigmas complementarios. Se evidenció que no existe un paradigma de comunicación universalmente superior: la decisión correcta depende de los requisitos específicos de cada flujo de datos, particularmente del balance entre confiabilidad y latencia. El uso de la API de Sockets de Java como middleware de transporte, junto con un `ExecutorService` como middleware de concurrencia, demostró ser una base sólida y portable para la prueba de concepto, quedando como trabajo futuro la incorporación efectiva de un servicio de RPC y de un *broker* de mensajería dentro de la arquitectura objetivo.

13. Referencias Bibliográficas

Referencias

- [1] G. Coulouris, J. Dollimore, T. Kindberg, and G. Blair, *Distributed Systems: Concepts and Design*, 5th ed. Boston, MA, USA: Addison-Wesley, 2011.
- [2] A. S. Tanenbaum and M. Van Steen, *Distributed Systems: Principles and Paradigms*, 2nd ed. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2007.